

数据库系统 Database System

Instructor: Yiwen Shen, Ph.D.

Dept. of Software & Computer Engineering,

Ajou University, Korea

chrisshen@ajou.ac.kr

主题

- 复杂数据类型
- 应用开发

复杂数据类型

Complex Data Types

半结构化数据

- 许多应用程序需要存储复杂的数据，而这些数据的模式经常会发生变化。
- 关系模型对原子数据类型的要求可能有点严格了。
 - 例如，将兴趣集合存储为用户个人资料的集合值属性可能比对其进行规范化更简单。
- **半结构化数据**能极大地促进数据交换。
 - 数据交换可以在应用程序之间进行，也可以在应用程序的后端和前端之间进行。
 - 如今，Web 服务被广泛应用，复杂的数据被获取到前端，并通过移动应用或 JavaScript 显示出来。
- **JSON** 和 **XML** 是广泛使用的**半结构化数据模型**。

半结构化数据模型的特点

- 灵活的模式

- 宽列 表示方式：允许每个元组拥有不同的属性集，并且可以随时添加新属性
- 稀疏列 表示方法：模式具有固定但庞大的属性集，但每个元组可能仅存储其中的一个子集。

- 多值数据类型

- 集合，多重集合

- 例如：兴趣集合 {'篮球', '西甲联赛', '烹饪', '动漫', '爵士乐'}

- 键值映射（或简称 映射, **Key-value map**）

- 存储一组键值对
- 例如，{(brand, Apple), (ID, MacBook Air), (size, 13), (color, silver)}
- 映射操作： *put(key, value)*, *get(key)*, *delete(key)*

半结构化数据模型的特点

- 数组

- 广泛应用于科学研究和监测领域
- 例如，定期采集的读数可以用值数组表示，而不是表示成 (时间, 值) 对。
 - 用 [5, 8, 9, 11] 代替 {(1,5), (2, 8), (3, 9), (4, 11)}

- 多值属性类型

- 采用 *非第一范式* (*NFNF*) 数据模型进行建模
- 目前大多数数据库系统都支持此功能。

- 数组数据库：一种专门为数组提供支持的数据库。

- 例如，压缩存储、查询语言扩展等。
- Oracle GeoRaster、PostGIS、SciDB等

嵌套数据类型

- 分层数据在许多应用中都很常见
- **JSON: JavaScript 对象表示法 (JavaScript Object Notation)**
 - 当今被广泛使用
- **XML: 可扩展标记语言 (Extensible Markup Language)**
 - 更早时代的记录方法，至今仍被广泛使用

JSON

- 是一种广泛用于数据交换的文本表示法
- JSON 数据示例

```
{  
  "ID": "22222",  
  "name": {  
    "firstname": "Albert",  
    "lastname": "Einstein"  
  },  
  "deptname": "Physics",  
  "children": [  
    { "firstname": "Hans", "lastname": "Einstein" },  
    { "firstname": "Eduard", "lastname": "Einstein" }  
  ]  
}
```

- 类型：整数、实数、字符串和
 - 对象：是键值映射，即（属性名称，值）对的集合。
 - 数组 也是键值映射（从偏移量到值）。

JSON

- 如今JSON在数据交换中无处不在。
 - 广泛用于网络服务
 - 大多数现代应用程序都是围绕 Web 服务构建的。
- SQL 扩展
 - 用于存储 JSON 数据的 JSON 类型
 - 使用路径表达式 (path expressions) 从 JSON 对象中提取数据
 - 例如 $V \rightarrow ID$, 或 $v.ID$
 - 从关系数据生成 JSON
 - 例如 `json.build_object('ID', 12345, 'name', 'Einstein')`
 - 使用聚合创建 JSON 集合
 - 例如 PostgreSQL 中的 `json_agg` 聚合函数。
 - 不同数据库的语法差异很大。
- JSON 信息冗长
 - 压缩表示形式, 例如 BSON (二进制 JSON) , 用于高效数据存储。

XML

- XML 使用标签来标记文本
- 例如

```
<course>  
  <course id> CS-101 </course id>  
  <title> Intro. to Computer Science </title>  
  <dept name> Comp. Sci. </dept name>  
  <credits> 4 </credits>  
</course>
```
- 标签使数据具有自记录性
- 标签可以是层级式的。

XML 数据示例

```
<purchase order>
  <identifier> P-101 </identifier>
  <purchaser>
    <name> Cray Z. Coyote </name>
    <address> Route 66, Mesa Flats, Arizona 86047, USA </address>
  </purchaser>
  <supplier>
    <name> Acme Supplies </name>
    <address> 1 Broadway, New York, NY, USA </address>
  </supplier>
  <itemlist>
    <item>
      <identifier> RS1 </identifier>
      <description> Atom powered rocket sled </description>
      <quantity> 2 </quantity>
      <price> 199.95 </price>
    </item>
    <item>...</item>
  </itemlist>
  <total cost> 429.85 </total cost>
  ...
</purchase order>
```

XML 续

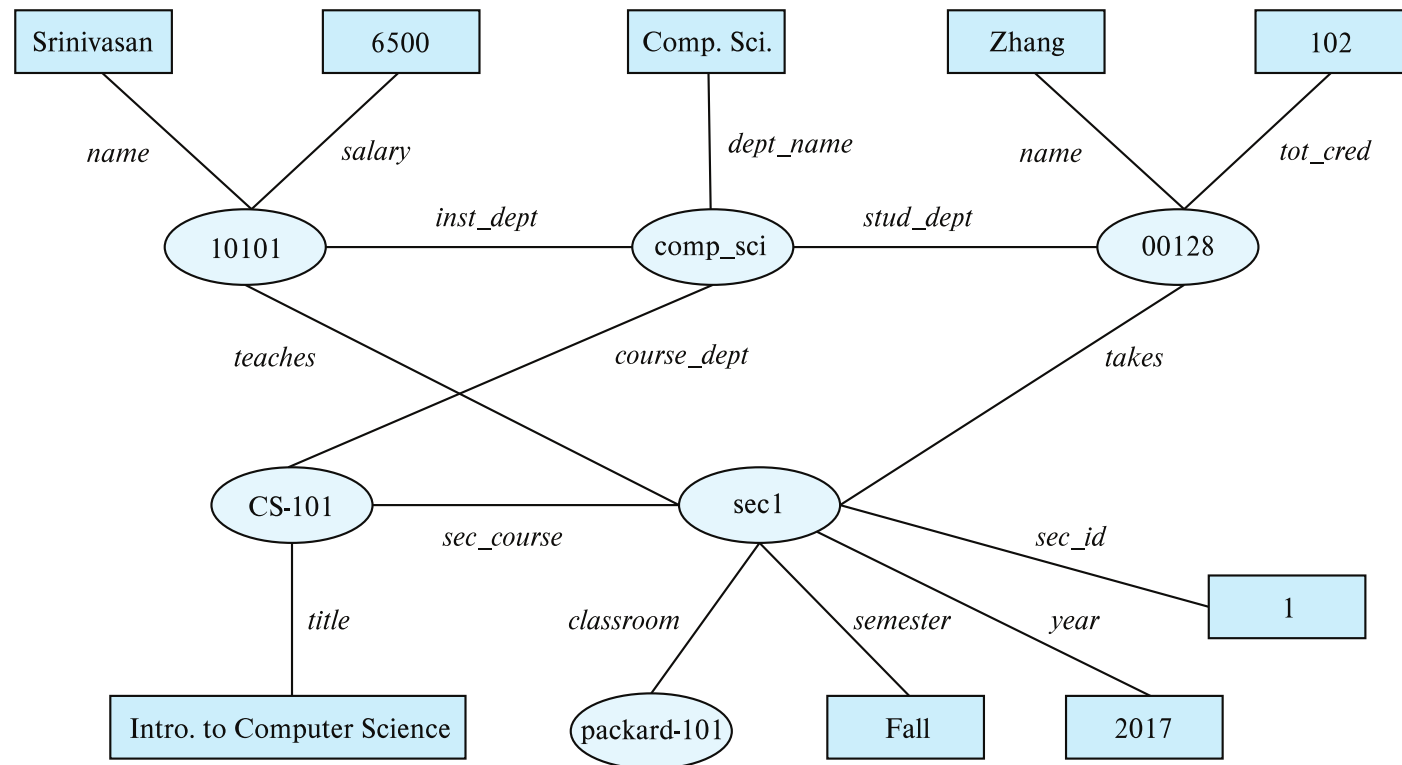
- **XQuery 语言是为查询嵌套 XML 结构而开发的。**
 - 目前尚未广泛使用。
- **支持 XML 的 SQL 扩展**
 - 存储 XML 数据
 - 从关系数据生成 XML 数据
 - 从 XML 数据类型中提取数据
 - 路径表达式

知识表示

- 人类知识的表征一直是人工智能的长期目标。
 - 随着时间的推移，人们提出了各种事实表述和推理规则。
- **RDF: 资源描述格式 (Resource Description Format)**
 - 事实的简化表示，以三元组 (*主语*、*谓语*、*宾语*) 的形式表示。
(*subject, predicate, object*)
 - 例如， (NBA-2019, 冠军, 猛龙队)
(华盛顿特区, 首都, 美国)
(华盛顿特区, 人口, 6,200,000)
 - 模型对象具有属性以及与其他对象的关系
 - 类似于 ER 模型，但具有灵活的模式
 - (*ID*, 属性名称, *value*)
 - (*ID1*, 关系名称, *ID2*)
 - 具有自然的图表示

RDF数据的图形视图

- 知识图谱



RDF数据的三重视图

10101	instance-of	instructor .
10101	name	"Srinivasan" .
10101	salary	"6500" .
00128	instance-of	student .
00128	name	"Zhang" .
00128	tot_cred	"102" .
comp_sci	instance-of	department .
comp_sci	dept_name	"Comp. Sci." .
biology	instance-of	department .
CS-101	instance-of	course .
CS-101	title	"Intro. to Computer Science" .
CS-101	course_dept	comp_sci .
sec1	instance-of	section .
sec1	sec_course	CS-101 .
sec1	sec_id	"1" .
sec1	semester	"Fall" .
sec1	year	"2017" .
sec1	classroom	packard-101 .
sec1	time_slot_id	"H" .
10101	inst_dept	comp_sci .
00128	stud_dept	comp_sci .
00128	takes	sec1 .
10101	teaches	sec1 .

查询 RDF: SPARQL

- 三重模式
 - `?cid title "Intro. to Computer Science"`
 - `?cid title "Intro. to Computer Science"`
`?sid course ?cid`
- SPARQL 查询
 - ```
select ?name
where {
 ?cid title "Intro. to Computer Science" .
 ?sid course ?cid .
 ?id takes ?sid .
 ?id name ?name .
}
```
  - 也支持
    - 聚合、可选连接（类似于外连接）、子查询等。
    - 路径上的传递闭包



# RDF 表示 (续)

- RDF三元组表示二元关系
- 如何表示n元关系?
  - 方法一：创建人造实体，并将其链接到 n 个实体中的每一个。
    - 例如, (Barack Obama, *president-of*, USA, 2008-2016) 可以表示为  
(*e1*, *person*, Barack Obama), (*e1*, *country*, USA),  
(*e1*, *president-from*, 2008) (*e1*, *president-till*, 2016)
  - 方法二：使用四元组而非三元组，并带有上下文实体
    - 例如: (Barack Obama, *president-of*, USA, *c1*)  
(*c1*, *president-from*, 2008) (*c1*, *president-till*, 2016)
- RDF被广泛用作知识库表示方法。
  - DBPedia 、 Yago 、 Freebase、 WikiData 、 ..
- 关联开放数据 该项目旨在连接不同的知识图谱，以允许跨数据库查询。

# 面向对象

- **对象关系数据模型**提供更丰富的类型系统
  - 具有复杂数据类型和面向对象特性
- 应用程序通常使用面向对象的编程语言编写。
  - 类型系统与关系类型系统不匹配
  - 在命令式语言和 **SQL** 之间切换很麻烦
- 将面向对象与数据库集成的方法
  - 构建**对象关系数据库**，向关系数据库添加面向对象特性
  - 自动在编程语言模型和关系模型之间转换数据；数据转换由**对象关系映射**指定。
  - 构建一个原生支持面向对象数据并可从编程语言直接访问的**面向对象数据库**

# 对象关系数据库系统

- 用户自定义类型

```
create type Person
(ID varchar(20) primary key,
 name varchar(20),
 address varchar(20)) ref from(ID);
```

```
create table people of Person;
```

- 表格类型

```
create type interest as table (
 topic varchar(20),
 degree_of_interest int);
```

```
create table users (
 ID varchar(20),
 name varchar(20),
 interests interest);
```

- 许多数据库也支持数组和多重集数据类型。
  - 语法因数据库而异

# 类型和表继承

- 类型继承

```
create type Student under Person
(degree varchar(20)) ;
create type Teacher under Person
(salary integer);
```

- PostgreSQL 和 Oracle 中的表继承语法

```
create table students
(degree varchar(20))
inherits people;
create table teachers
(salary integer)
inherits people;
create table people of Person;
create table students of Student
under people;
create table teachers of Teacher
under people;
```

# 参考类型

- 创建引用类型

```
create type Person
 (ID varchar(20) primary key,
 name varchar(20),
 address varchar(20))
 ref from(ID);
create table people of Person;
create type Department (
 dept_name varchar(20),
 head ref(Person) scope people);
create table departments of Department
insert into departments values ('CS', '12345')
• 可以使用子查询检索系统生成的参考文献。
 (select ref(p) from people as p where ID = '12345')
```

- 路径表达式中使用引用

- select *head->name*, *head->address*  
from *departments*;

# 对象关系映射

- **对象关系映射 (ORM, Object-relational mapping) 系统允许**
  - 编程语言对象与数据库元组之间映射关系的规范
  - 创建对象时自动创建数据库元组
  - 当对象更新/删除时，数据库元组自动更新/删除
  - 用于检索满足指定条件的对象的接口
    - 查询数据库中的元组，并根据这些元组创建对象。
- **细节**
  - **Hibernate ORM for Java**
  - **用于 Python 的 Django ORM**

# 文本数据

- **信息检索**：对非结构化数据进行查询
  - 关键词查询的简单模型：给定查询关键词，检索包含所有关键词的文档。
  - 更高级的模型会对文档的相关性进行排名。
  - 如今，关键词查询会返回多种类型的信息作为答案。
    - 例如，查询“板球”通常会返回正在进行的板球比赛信息。
- **相关性排名**
  - 这一点至关重要，因为通常有很多文档与关键词匹配。

# 使用**TF-IDF**进行排名

- 词项：文档/查询 中出现的关键词
- **词频**：  $TF(d, t)$ ，表示词项  $t$  与文档  $d$  的相关性。
  - 定义：  $TF(d, t) = \log(1 + n(d, t) / n(d))$   
其中
    - $n(d, t)$  = 术语  $t$  在文档  $d$  中出现的次数
    - $n(d)$  = 文档  $d$  中的词项数量
- **逆文档频率**：  $IDF(t)$ 
  - 一种定义方式：  $IDF(t) = 1 / n(t)$
- 文档  $d$  映射 到 一组术语  $Q$  的关联性
  - 定义：  $r(d, Q) = \sum_{t \in Q} TF(d, t) * IDF(t)$
  - 其他定义
    - 考虑词语的**邻近性**
    - **停顿词** 经常被忽视

$$\text{idf}(t, D) = \log \frac{N}{n_t}$$



# 利用超链接进行排名

- 超链接为理解重要性提供了非常重要的线索。
- 谷歌推出了 PageRank，这是一种基于页面超链接的受欢迎程度/重要性衡量指标。
  - 从许多页面获得超链接的页面应该具有更高的 PageRank。
  - 从 PageRank 值较高的页面链接到的页面，其 PageRank 值也应该较高。
  - 随机游走模型形式化
- 令  $T[i, j]$  表示位于第  $i$  页的随机游走者 点击 指向 第  $j$  页链接的概率。
  - 假设所有链路都相等，则  $T[i, j] = 1/N_i$
- 那么，每个页面  $j$  的 PageRank[j] 可以定义为
  - $P[j] = \delta/N + (1 - \delta) * \sum_{i=1}^N (T[i, j] * P[i])$
  - 其中  $N$  = 总页数， $\delta$  为常数，通常设为 0.15。

# 利用超链接进行排名

- PageRank的定义是循环的，但可以通过求解一组线性方程组来解决。
  - 简单的迭代方法效果很好。
  - 初始化所有  $P[i] = 1/N$
  - 在每次迭代中，使用公式  $P[j] = \delta/N + (1 - \delta) * \sum_{i=1}^N (T[i, j] * P[i])$  来更新  $P$
  - 当变化很小，或者达到某个限制（例如 30 次迭代）时，停止迭代。
- 其他相关性指标也很重要。例如：
  - 锚文本 (anchor text) 中的关键词
  - 如果问题的答案是“已解答”，则提问者点击链接的次数。

# 检索有效性

- 有效性指标

- **精确度 (Precision)**: 返回结果中实际相关的百分比。
- **召回率 (Recall)**: 返回的相关结果占比是多少?
- 在达到一定数量的答案时, 例如精确率@10, 召回率@10

- 结构化数据和知识库的关键词查询

- 如果用户不了解模式, 或者没有预定义的模式, 则此方法很有用。
- 可以将数据表示为图表。
- 关键词匹配元组
- 关键词搜索返回包含关键词的紧密关联的元组
  - 例如, 在我们大学数据库中, 如果查询“Zhang Katz”, 则 Zhang 匹配一名学生, Katz 匹配一名教师和导师, 两者之间存在关联。

# 空间数据

# 空间数据

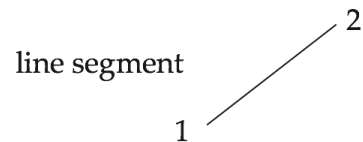
- 空间数据库存储与空间位置相关的信息，并支持空间数据的高效存储、索引和查询。
  - **地理数据**——道路地图、土地利用地图、地形高程地图、显示边界的政治地图、土地所有权地图等等。
    - **地理信息系统**是专门用于存储地理数据的数据库。
    - 可以使用球形地球坐标系。
      - (纬度、经度、海拔)
  - **几何数据**: 描述物体构造方式的设计信息。例如，建筑物设计图、飞机设计图、集成电路布局图。
    - 具有 (X, Y, Z) 坐标的二维或三维欧几里得空间

# 几何信息的表示

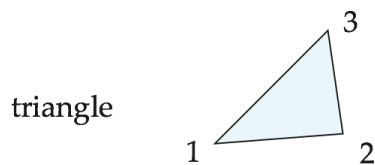
各种几何结构可以以规范化的方式在数据库中表示(见下一张幻灯片)

- **线段 (line segment)** 可以用其端点的坐标来表示。
- **折线或线串** 由一系列连接的线段组成，可以用一个列表来表示，该列表按顺序包含线段端点的坐标。
  - 通过将曲线分割成一系列线段来近似曲线。
    - 适用于道路等二维特征。
    - 有些系统也支持将 **圆弧** 作为基本图形元素，从而允许将曲线表示为一系列圆弧。
- **多边形** 用按顺序排列的**顶点列表**表示。
  - 顶点列表指定了多边形区域的边界。
  - 也可以表示为一组三角形(**三角剖分**)

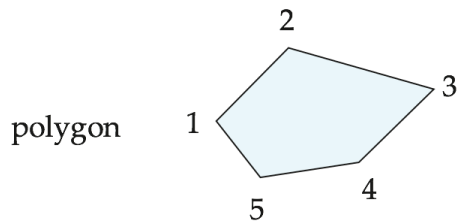
# 几何结构的表示



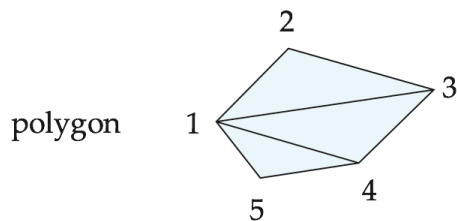
$\{(x1,y1), (x2,y2)\}$



$\{(x1,y1), (x2,y2), (x3,y3)\}$



$\{(x1,y1), (x2,y2), (x3,y3), (x4,y4), (x5,y5)\}$



$\{(x1,y1), (x2,y2), (x3,y3), ID1\}$   
 $\{(x1,y1), (x3,y3), (x4,y4), ID1\}$   
 $\{(x1,y1), (x4,y4), (x5,y5), ID1\}$

**object**

**representation**

# 几何信息的表示 (续)

- 三维空间中点和线段的表示方法与二维类似，区别在于三维空间中的点多了一个  $z$  分量。
- 任意多面体分割成四面体来表示它们，就像对多边形进行三角剖分一样。
- 另一种方法：列出它们的面，每个面都是一个多边形，并指出面的哪一面位于多面体内部。
- 许多数据库支持的几何和地理数据类型
  - 例如 SQL Server 和 PostGIS
  - 点、线串、曲线、多边形 (point, linestring, curve, polygons)
  - 集合：多点、多线串、多曲线、多边形
  - LINESTRING(1 1, 2 3, 4 4)
  - POLYGON((1 1, 2 3, 4 4, 1 1))
  - 类型转换： *ST GeometryFromText()* 和 *ST GeographyFromText()*
  - 操作： *ST Union()*、 *ST Intersection()*、 ...

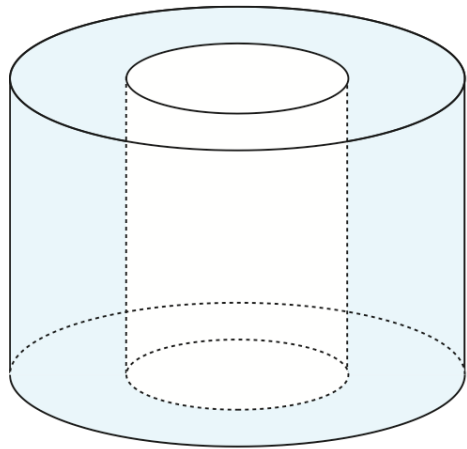


# 设计数据库

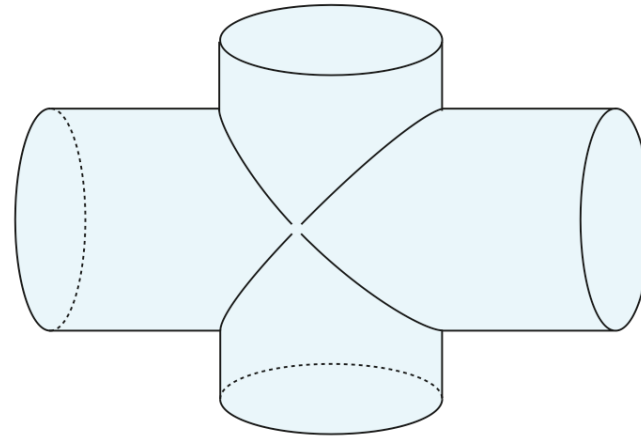
- 将设计组件表示为对象（通常是几何对象）；对象之间的连接表明了设计的结构方式。
- 简单的二维物体：点、线、三角形、矩形、多边形。
- 复杂的二维物体：由简单物体通过并集、交集和差集运算形成。
- 复杂的三维物体：由球体、圆柱体和长方体等较简单的物体通过并集、交集和差集运算形成。
- 线框模型将三维曲面表示为一组更简单的对象。

# 几何结构的表示

- 设计数据库还会存储有关对象的非空间信息（例如，建筑材料、颜色等）。
- 空间完整性约束非常重要。
  - 例如，管道不应交叉，电线不应靠得太近等等。



(a) Difference of cylinders



(b) Union of cylinders

# 地理数据

- **栅格数据** 由二维或多维的位图或像素图组成。
  - 二维栅格图像示例：云覆盖的卫星图像，其中每个像素存储特定区域的云可见性。
  - 其他维度可能包括不同地区不同海拔的温度，或者不同时间点的测量值。
- 设计数据库通常不存储栅格数据。

# 地理数据 (续)

- **向量数据** 图形由基本几何对象构成：二维图形由点、线段、三角形和其他多边形构成，三维图形由圆柱体、球体、长方体和其他多面体构成。
- 矢量格式常用于表示地图数据。
  - 道路可以看作是二维的，可以用直线和曲线来表示。
  - 有些地物，例如河流，可以根据其宽度是否相关，用复杂的曲线或复杂的多边形来表示。
  - 区域和湖泊等地物可以用多边形来表示。

# 空间查询

- **区域查询** 处理空间区域。例如，查找部分或完全位于指定区域内的对象。
  - 例如， PostGIS *ST\_Contains ()* 、 *ST\_Overlaps ()* 、 ...
- **邻近性查询** 请求位于指定位置附近的对象。
- **最近邻查询**， 给定一个点或一个对象， 找到满足给定条件的最近对象。
- **空间图查询**请求基于空间图的信息
  - 例如， 道路网络中两点之间的最短路径
- **空间连接** 两个空间关系， 位置作为连接属性。
- 计算区域交集或**并集的查询**

# 应用开发

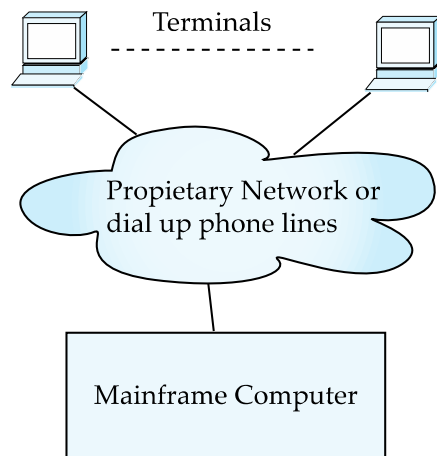
Application Development

# 应用程序和用户界面

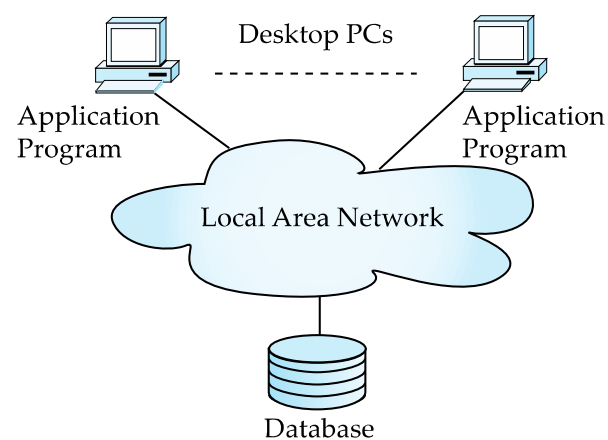
- 大多数数据库用户不使用像 SQL 这样的查询语言。
- 应用程序充当用户和数据库之间的中介。
  - 应用程序分为
    - 前端
    - 中间层
    - 后端
- 前端：用户界面
  - 表格
  - 图形用户界面
  - 许多界面都是基于Web的。

# 应用架构演进

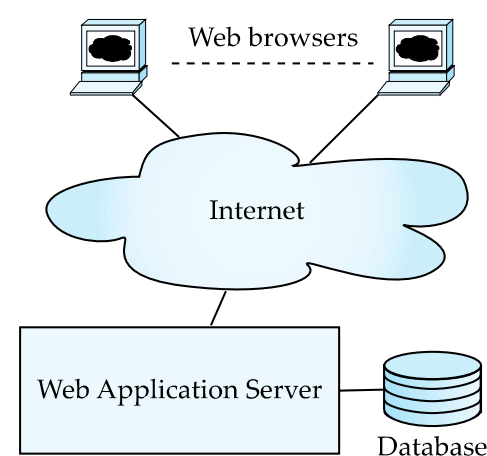
- 应用架构的三个不同时代
  - 大型机(20世纪60年代和70年代)
  - 个人电脑时代(20世纪80年代)
  - 互联网时代(20世纪90年代中期至今)
  - 互联网和智能手机时代(2010年至今)



(a) Mainframe Era



(b) Personal Computer Era  
2025年秋季



(c) Web era



# 网页界面

网络浏览器已成为数据库事实上的标准用户界面。

- **允许大量用户从任何地点访问数据库**
- **无需下载/安装专用代码，同时提供良好的图形用户界面**
  - Javascript、Flash 和其他脚本语言在浏览器中运行，但下载过程是透明的。
- **例如：银行、航空公司和租车公司的预订、大学课程注册和评分等等。**

# 示例 HTML 源代码

`<html>`

`<body>`

`<table border>`

`<tr> <th>ID</th> <th>Name</th> <th>Department</th> </tr>`

`<tr> <td>00128</td> <td>Zhang</td> <td>Comp. Sci.</td> </tr>`

`....`

`</table>`

`<form action="PersonQuery" method=get>`

Search for:

`<select name="persontype">`

`<option value="student" selected>Student </option>`

`<option value="instructor"> Instructor </option>`

`</select> <br>`

Name: `<input type=text size=20 name="name">`

`<input type=submit value="submit">`

`</form>`

`</body> </html>`

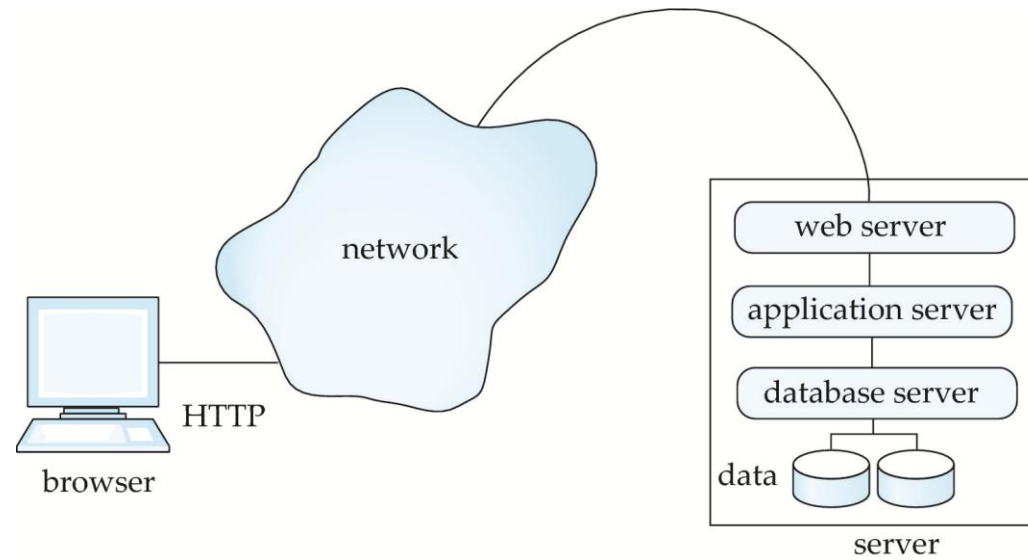
# 显示示例 HTML 源代码

| ID    | Name    | Department |
|-------|---------|------------|
| 00128 | Zhang   | Comp. Sci. |
| 12345 | Shankar | Comp. Sci. |
| 19991 | Brandt  | History    |

Search for:

Name:

# 三层Web架构



# HTTP 和会话 (Sessions)

- HTTP 协议是**无连接的**。
  - 也就是说, 服务器一旦响应了请求, 就会关闭与客户端的连接, 并彻底忘记该请求。
  - 相比之下, Unix 登录和 JDBC/ODBC 连接会一直保持连接状态, 直到客户端断开连接为止。
    - 保留用户身份验证和其他信息
  - 动机: **减轻服务器负载**
    - 操作系统对机器上打开的连接数有严格的限制。
- 信息服务需要会话信息
  - 例如, 用户身份验证每个会话只需进行一次。
- 解决方案: 使用**Cookie**

# 会话和 Cookie

- **Cookie**是一小段包含识别信息的代码。
  - 服务器发送给浏览器
    - 首次互动时发送, 用于识别会话
  - 浏览器会在后续交互中将此信息发送到创建该 cookie 的服务器。
    - HTTP协议的一部分
  - 服务器会保存有关其发出的 cookie 的信息, 并可在处理请求时使用这些信息。
    - 例如, 身份验证信息和用户偏好
- **Cookie** 可以永久存储, 也可以在有限的时间内存储。

# Servlet

- **Java Servlet规范定义了一个API，用于Web/应用程序服务器和服务器上运行的应用程序之间进行通信。**
  - 例如，从 Web 表单获取参数值以及将 HTML 文本发送回客户端的方法。
- **应用程序（也称为 servlet）被加载到服务器中**
  - **每个请求都会在服务器上创建一个新线程。**
    - 请求处理完毕后，线程即关闭。
  - **程序员创建了一个继承自HttpServlet 的类**
    - 并重写方法doGet、doPost等
  - **servlet 名称(可通过 HTTP 访问)到 servlet 类的映射是在 web.xml 文件中完成的。**
    - 大多数 IDE 在您使用 IDE 创建 Servlet 时会自动完成此操作。

# Servlet 示例代码

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class PersonQueryServlet extends HttpServlet {
 public void doGet (HttpServletRequest request, HttpServletResponse response)
 throws ServletException, IOException
 {
 response.setContentType("text/html");
 PrintWriter out = response.getWriter();
 out.println("<HEAD><TITLE> Query Result</TITLE></HEAD>");
 out.println("<BODY>");
 BODY OF SERVLET (next slide) ...
 out.println("</BODY>");
 out.close();
 }
}
```



# Servlet 示例代码

```
String persontype = request.getParameter("persontype");
String number = request.getParameter("name");
if(persontype.equals("student")){
 ... code to find students with the specified name ...
 ... using JDBC to communicate with the database ..
 out.println("<table BORDER COLS=3>");
 out.println(" <tr> <td>ID</td> <td>Name: </td>" + " <td>Department</td> </tr>");
 for(... each result ...){
 ... retrieve ID, name and dept name
 ... into variables ID, name and deptname
 out.println("<tr> <td>" + ID + "</td>" + "<td>" + name + "</td>" + "<td>" + deptname
 + "</td></tr>");
 };
 out.println("</table>");
}
else {
 ... as above, but for instructors ...
}
```

# Servlet 会话

- **Servlet API 支持会话处理**
  - 首次与浏览器交互时设置 cookie, 并使用该 cookie 在后续交互中识别会话。
- **检查会话是否已处于活动状态:**
  - 如果 ( `request.getSession (false) == true` )
    - 然后是现有会话
    - 否则, 重定向到身份验证页面
  - **身份验证页面**
    - 检查登录名/密码
    - 创建新会话
      - `HttpSession session = request.getSession (true)`
    - 存储/检索特定会话的属性值对
      - `session.setAttribute ( "userid", userid )`
  - 如果会话已存在:
    - `HttpSession = request.getSession (false);`
    - `String userid = (String) session.getAttribute ( "userid " )`

# Servlet 支持

- **Servlet运行在应用服务器内部，例如**
  - **Apache Tomcat、Glassfish、JBoss**
  - **BEA Weblogic 、IBM WebSphere和 Oracle 应用服务器**
- **应用服务器支持**
  - **Servlet 的部署和监控**
  - **Java 2 企业版 (J2EE) 平台支持对象、跨多个应用服务器的并行处理等功能。**

# 服务器端脚本

- 服务器端脚本简化了将数据库连接到 Web 的任务。
  - 定义一个包含嵌入式可执行代码/SQL查询的HTML文档。
  - 来自 HTML 表单的输入值可以直接用于嵌入式代码/SQL 查询。
  - 当请求该文档时, Web 服务器执行嵌入的代码/SQL 查询来生成实际的 HTML 文档。
- 多种服务器端脚本语言
  - JSP、PHP
  - 通用脚本语言: VBScript、Perl、Python

# Java 服务器页面 (JSP)

- 一个嵌入了 Java 代码的 JSP 页面

```
<html>
```

```
<head> <title> Hello </title> </head>
```

```
<body>
```

```
<% if (request.getParameter("name") == null)
```

```
{ out.println("Hello World"); }
```

```
else { out.println("Hello, " + request.getParameter("name")); }
```

```
%>
```

```
</body>
```

```
</html>
```

- JSP 被编译成 Java + Servlet。
- JSP 允许在标签库中定义新标签。
  - 这类标签就像库函数一样, 例如可以用来构建丰富的用户界面, 如大型数据集的分页显示。

# PHP

- PHP广泛用于Web服务器脚本编写。
- 包含用于使用 ODBC 进行数据库访问的大量库

**<html>**

**<head> <title> Hello </title> </head>**

**<body>**

**<?php if (!isset(\$\_REQUEST['name']))**

**{ echo "Hello World"; }**

**else { echo "Hello, " + \$\_REQUEST['name']; }**

**?>**

**</body>**

**</html>**

# Javascript 前端脚本语言

- Javascript被广泛使用
  - 构成新一代Web应用程序(称为Web 2.0应用程序)的基础, 提供丰富的用户界面
- Javascript函数可以
  - 检查输入内容的有效性
  - **文档对象模型(DOM)**来修改显示的网页。显示的 HTML 文本的树状表示
  - 与Web服务器通信以获取数据, 并使用获取的数据修改当前页面, 而无需重新加载/刷新页面。
    - 它构成了广泛应用于 Web 2.0 应用的 AJAX 技术的基础。
    - 例如, 在下拉菜单中选择一个国家后, 该国家/地区的州/省列表会自动填充到关联的下拉菜单中。

# Javascript

- 是使用JavaScript验证表单输入的示例

```
<html> <head>
 <script type="text/javascript">
 function validate() {
 var credits=document.getElementById("credits").value;
 if (isNaN(credits)|| credits<=0 || credits>=16) {
 alert("Credits must be a number greater than 0 and less than 16");
 return false
 }
 }
 </script>
</head> <body>
 <form action="createCourse" onsubmit="return validate()">
 Title: <input type="text" id="title" size="20">

 Credits: <input type="text" id="credits" size="2">

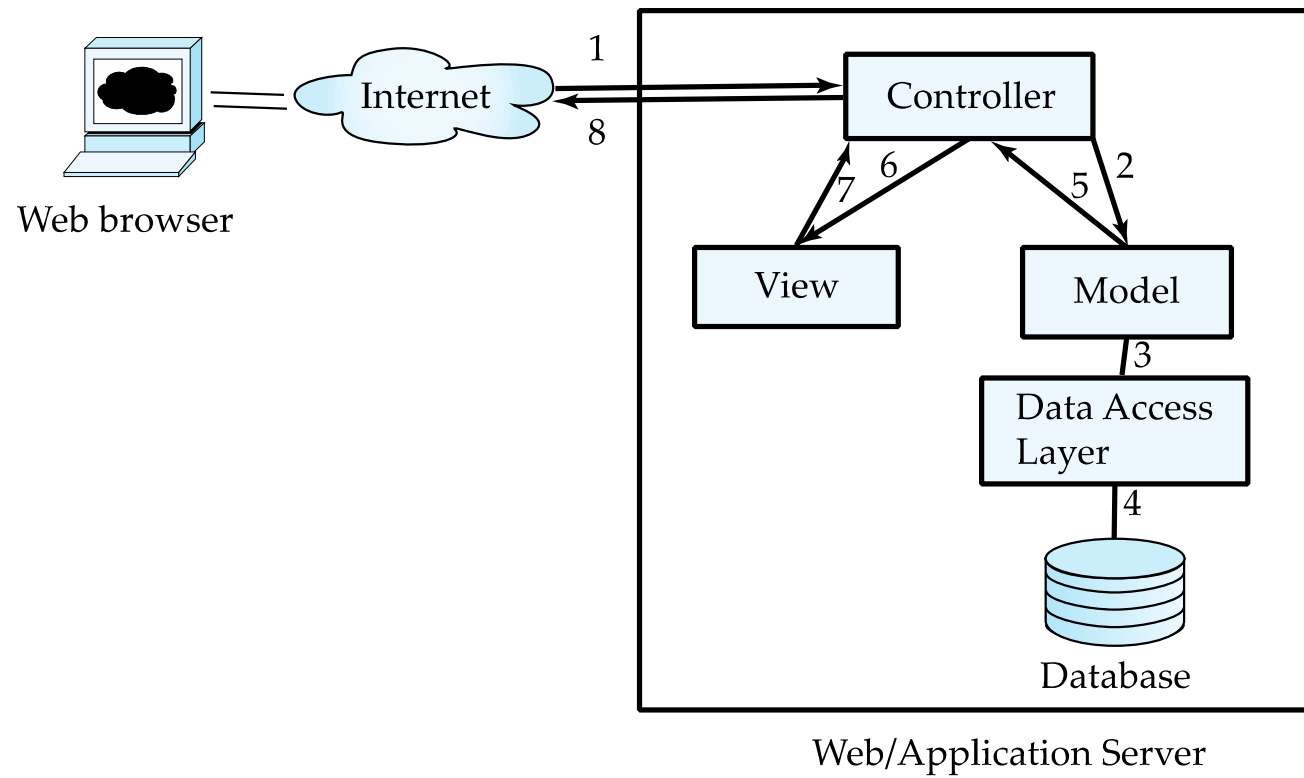
 <Input type="submit" value="Submit">
 </form>
</body> </html>
```



# 应用架构

- 应用层
  - 演示或用户界面
    - **模型-视图-控制器(MVC)** 架构
      - **模型**: 业务逻辑
      - **视图**: 数据的呈现方式, 取决于显示设备
      - **控制器**: 接收事件、执行操作并向用户返回数据视图
  - **业务逻辑层**
    - 提供数据和数据操作的高级概览
      - 通常使用对象数据模型
    - 隐藏数据存储方案的详细信息
  - **数据访问层**
    - 业务逻辑层与底层数据库之间的接口
    - 提供从业务层对象模型到数据库关系模型的映射

# 应用架构



# 业务逻辑层

- 提供实体的抽象
  - 例如:学生、教师、课程等
- 执行**业务规则** 用于执行行动
  - 先修课程并缴纳学费后才能选修课程。
- 支持**工作流程** 它定义了如何执行涉及多名参与者的任务。
  - 例如, 如何处理学生向大学提交的申请。
  - 执行任务的步骤顺序
  - 错误处理
    - 例如, 如果推荐信没有按时收到该怎么办?

# 对象关系映射

- 允许在面向对象数据模型之上编写应用程序代码，同时将数据存储在传统的关系数据库中。
  - 另一种方法:实现面向对象或对象关系数据库来存储对象模型
    - 商业上并不成功。
- 模式设计者必须提供对象数据和关系模式之间的映射。
  - 例如, Java 类 *Student* 映射到关系 *student*, 并具有相应的属性映射。
  - 一个对象可以映射到多个关系中的多个元组。
- 应用程序打开一个会话, 该会话连接到数据库。
- **session.save (object)** 创建对象并将其保存到数据库中。
  - 映射用于在数据库中创建相应的元组
- 可以运行查询来检索满足指定谓词的对象

# 对象关系映射 (ORM) 和 Hibernate (续)

- **Hibernate** 对象关系映射系统 被广泛使用。
  - 公共领域系统, 可在多种数据库系统上运行
  - 支持一种能够表达涉及连接的复杂查询的查询语言。
    - 将查询转换为 SQL 查询
  - 允许将关系映射到与对象关联的集合。
    - 例如, 学生所修课程可以作为 Student 对象中的一个集合。
- 由微软开发的实体**数据模型**
  - 直接向应用程序提供实体关系模型
  - 在实体数据模型和底层存储(可以是关系型存储)之间映射数据。
  - 实体 SQL 语言直接操作实体数据模型

# 网络服务

- 允许使用**远程过程调用 (remote procedure call) 机制**访问 Web 上的数据
- 两种方法被广泛采用
  - **表述性状态转移 (REST)** : 允许使用标准的 HTTP 请求向 URL 发送请求并返回数据。
    - 返回的数据以 XML 或 **JavaScript 对象表示法 (JSON)** 进行编码。
  - **大型网络服务** :
    - 使用 XML 表示形式发送请求数据, 以及返回结果。
    - 基于 HTTP 构建的标准协议层

# 断断续续的操作

- 用于使应用程序在连接网络时能够使用网络，而在断开网络连接时能够在本地运行的工具
  - 利用 HTML5 本地存储

# 快速应用开发

- 开发Web应用程序接口需要付出大量努力。
  - 此外, 还要支持与 Web 2.0 应用程序相关的丰富交互功能。
- 几种加快应用程序开发速度的方法
  - 用于生成用户界面元素的函数库
  - 在集成开发环境 (IDE) 中使用拖放功能创建用户界面元素
  - 根据声明式规范自动生成用户界面代码
- 早在Web出现之前就已被用作 **快速应用程序开发 (RAD) 工具**的一部分。
- Web应用程序开发框架
  - Java Server Faces (JSF) 包含 JSP 标签库
  - Ruby on Rails
    - 通过从数据库模式或对象模型生成代码, 轻松创建简单的**CRUD (创建、读取、更新和删除)**接口。



# 应用程序性能

# 提升Web服务器性能

- 性能是热门网站面临的一个问题
  - 每天可能有数百万用户访问, 高峰时段每秒请求数以千计。
- 缓存技术通过利用请求之间的共性来降低页面服务成本。
  - 在服务器端:
    - Servlet 请求之间的 JDBC 连接缓存
      - 又名 连接池
    - 缓存数据库查询结果
      - 如果底层数据库发生更改, 则必须更新缓存结果。
    - 缓存生成的 HTML
  - 在客户端网络上
    - 通过 Web 代理缓存页面

# 应用安全

# 跨站脚本攻击 Cross-Site Scripting

- 一个页面上的 HTML 代码在另一个页面上执行操作
  - 例如, `<img src = http://mybank.com/transfermoney?amount=1000&toaccount=14523>`
  - 风险提示: 如果浏览包含上述代码页面的用户当前已登录 mybank 账户, 则转账可能成功。
  - 上面的例子比较简单, 因为 GET 方法通常不用于更新, 但如果代码是脚本, 它就可以执行 POST 方法。
- 上述漏洞称为 **跨站脚本攻击 (XSS)** 或 **跨站请求伪造 (XSRF 或 CSRF) cross-site request forgery**
- 防止网站被用于发起 XSS 或 XSRF 攻击
  - 使用相关函数检测并移除用户输入文本中的 HTML 标签, 禁止在用户输入的文本中插入 HTML 标签。
- 保护网站免受来自其他网站的 XSS/XSRF 攻击

# 跨站脚本攻击

保护网站免受来自其他网站的 XSS/XSRF 攻击

- 使用由HTTP 协议提供的**引用页 (referrer)** 值( 链接被点击所在页面的 URL)用于检查该链接是否来自同一站点提供的有效页面, 而不是来自其他站点。
- 确保请求的 IP 地址与用户进行身份验证的 IP 地址相同。
  - 防止恶意用户劫持 cookie
- 切勿使用 GET 方法执行任何更新操作。
  - 这实际上是HTTP标准推荐的做法。

# 密码泄露

- 切勿将密码（例如数据库密码）以明文形式存储在用户可能访问的脚本中。
  - 例如，在可通过 Web 服务器访问的目录中的文件中
    - 通常情况下，Web 服务器会执行脚本，但不会提供诸如file.jsp或file.php之类的脚本文件的源代码，但可能会提供诸如file.jsp ~ 或file.jsp.swp之类的编辑器备份文件的源代码。
- 限制从运行应用程序服务器的机器的 IP 地址访问数据库服务器
  - 大多数数据库允许按源 IP 地址限制访问

# 应用程序身份验证

- **单因素 (single factor) 身份验证**（例如密码）对于关键应用程序来说风险过高
  - 如果密码未加密，则可以通过猜测密码和嗅探数据包来攻击攻击者。
  - 用户在不同网站重复使用密码
  - 窃取密码的间谍软件
- **双因素 (two-factor) 身份验证**
  - 例如，密码加上通过短信发送的一次性密码。
  - 例如，密码加一次性密码设备
    - 设备每分钟生成一个新的伪随机数，并显示给用户。
    - 用户输入当前号码作为密码
    - 应用服务器生成相同序列的伪随机数，以检查该数字是否正确。

# 应用程序身份验证

- **中间人**进攻
  - 例如, 一个伪装成 mybank.com 的网站, 将用户的请求转发给 mybank.com, 并将结果返回给用户。
  - 即使采用双因素身份验证也无法阻止此类攻击
- 解决方案: 使用**数字证书**和安全的HTTP协议 (https), 对网站用户进行身份验证。
- 在组织内部的 **中央认证**
  - 应用程序重定向到中央身份验证服务进行身份验证
  - 避免多个网站都能访问用户的密码
  - LDAP 或 Active Directory 用于身份验证



# 单点登录 Single Sign-On

- **单点登录**允许用户只需进行一次身份验证，应用程序即可与身份验证服务通信以验证用户身份，而无需重复输入密码。
- **安全断言标记语言 (SAML)** 用于跨安全域交换身份验证和授权信息的标准
  - 用户 ID 登录外部应用程序，例如 acm.org。 [joe@yale.edu](mailto:joe@yale.edu)
  - 该应用程序与耶鲁大学的基于 Web 的身份验证服务通信，以验证用户身份，并查找耶鲁大学授权用户执行的操作（例如，访问某些期刊）。
- **OpenID** 标准允许跨组织共享身份验证信息。
  - 例如，应用程序允许用户选择 Yahoo! 作为 OpenID 身份验证提供商，并将用户重定向到 Yahoo! 进行身份验证。

# 应用层授权

- 当前的 SQL 标准不允许细粒度的授权，例如“学生可以看到自己的成绩，但看不到其他学生的成绩”。
  - 问题 1: 数据库不知道哪些用户是应用程序用户
  - 问题 2: SQL 授权级别是表级别或表的列级别，而不是表的特定行级别。
- 一种解决方法：使用诸如视图之类的工具。

```
create view studentTakes as
 select *
 from takes
 where takes.ID = syscontext.user_id()
```

- 其中 *syscontext.user\_id()* 提供最终用户身份
  - 应用程序必须向数据库提供最终用户身份信息。
- 拥有多个这样的视图很麻烦

# 应用层授权 (续)

- 目前，授权完全在应用程序中完成。
- 整个应用程序代码都可以访问整个数据库。
  - 表面积大，使得防护难度增加
- 替代方案：细粒度（行级）授权 方案
  - 已提出 SQL 授权扩展方案，但目前尚未实现。
  - Oracle 虚拟专用数据库 (VPD) 允许以透明的方式向所有 SQL 查询添加谓词，从而强制执行细粒度的授权。
    - 例如，如果用户是学生，则在所有关于学生关系的查询中添加 `ID=sys_context.user_id()`。

# 审计跟踪 Audit Trails

- 应用程序必须将操作记录到**审计跟踪**中，以便检测谁执行了更新或访问了某些敏感数据。
- 事后使用的审计跟踪
  - 检测安全漏洞
  - 修复安全漏洞造成的损失
  - 追踪实施违规行为的人
- 需要审计跟踪
  - 数据库级别，以及在
  - 应用层

# 数据库系统

# Database System

## Thank you! Q/A?

Instructor: Yiwen Shen, Ph.D.

Dept. of Software & Computer Engineering,  
Ajou University, Korea